4. The Operating System Level

VIRTUAL MEMORY AND CACHES

Caches and Addressing

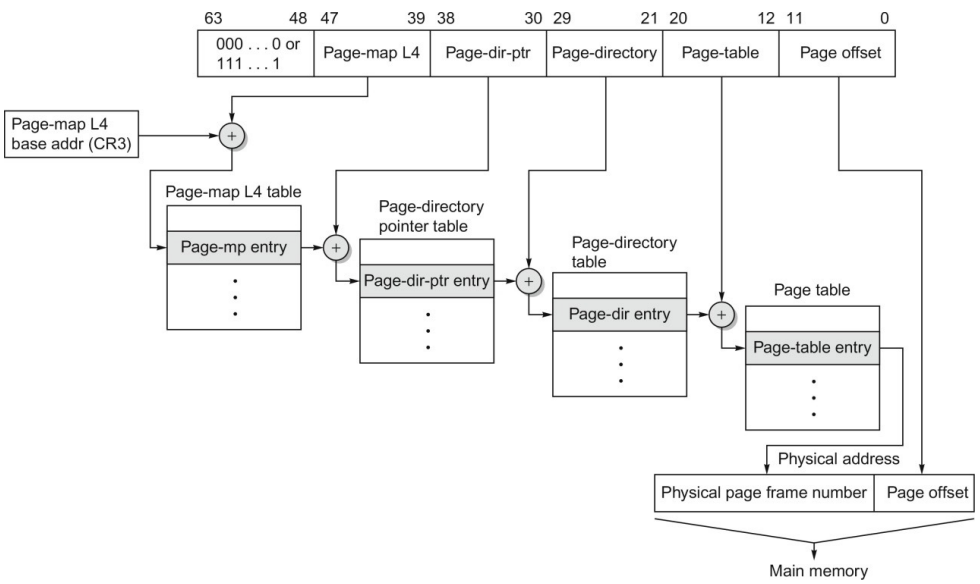
VA = <vt, vi, vf>

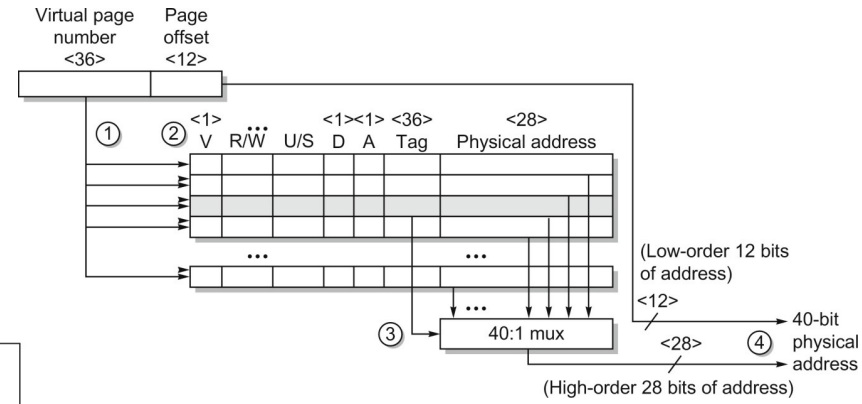
PA = <pt, pi, pf>

- The processor issues a virtual address (VA) in connection with a memory access.
- The memory subsystem deals with physical addresses (PA).
- The virtual memory subsystem in the OS performs $VA \rightarrow PA$ translation.
- Caches perform two functions to identify hit/miss:
  i. Indexing
  ii. Tag matching
- How does address translation interact with data caching?

	Virtual tag	Physical tag
Virtual index	VIVT	VIPT
Physical index	PIVT	PIPT

Page Tables and TLBs

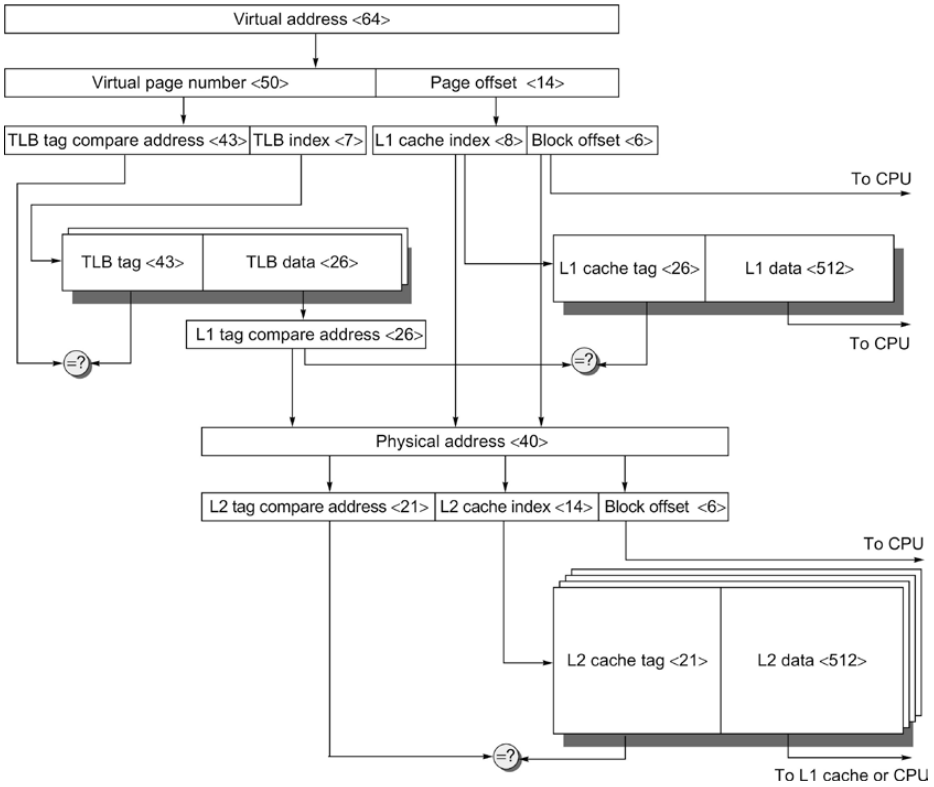




TLB Characteristic	L1 DTLB / ITLB	L2 STLB
Typical Size	32–128 entries	512–4096 entries
Access Latency	0.5–1 clock cycle	7–15 clock cycles
Associativity	High (often fully associative)	Medium (e.g., 8–16 way)
Specialization	Split (I/D)	Unified (Instruction + Data)

Translation/Data Access Interaction

- 64-bit VA, 40-bit PA.
- Page size 16 KiB.
- TLB $(2, 2^{14}, 2^{22};\ 2^{7})$.
- L1\$ $(1, 2^{6}, 2^{14})$.
  - VIPT
- L2\$ $(4, 2^{6}, 2^{22})$.
  - PIPT



Issues with Virtually-Indexed Caches

Homonyms and Synonyms

Homonyms

- *Definition*: Same name for two different pieces of data.
- *Cause*: VA used for indexing is context-dependent, and tag doesn't uniquely identify data.
- *Effect*: Wrong data is accessed.
- *Mitigation*:
  - Flush cache on context switch.
  - Force non-overlapping address-space layout.
  - Tag VA with address-space ID (ASID).
  - Use physical tags.

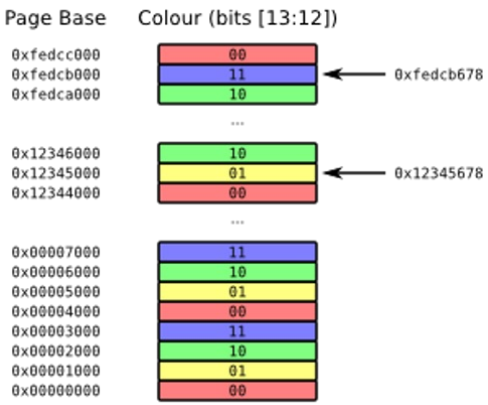
Synonyms

- *Definition*: Multiple names for the same piece of data.
- *Cause*: Several VA map to the same PA.
  - Frames shared between processes.
  - Multiple mappings of frame within AS.
- *Effect*: May access stale data.
  - Same data cached in multiple cache lines.
  - On write, one synonym is updated.
  - Read on the other synonym returns the old value.
- *Mitigation*:
  - Physical tags and ASIDs don't help.
  - Not an issue for RO data or I$.
  - For RW data, depends on page size and cache geometry.

Source: https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/page-colouring-on-armv6-and-a-bit-on-armv7

Synonyms, Page Size, Cache Geometry

VIPT Data Cache



- Suppose 4 KiB page size, 64-bit VAs, 16 GiB physical memory, and L1\$ $(A, 2^6, C;\ S)$.
  - # page frames = 2^{22}.
- `VA = {VPN, VPO}` with
  - `VPN = VA[63:12]`
  - `VPO = VA[11:0]`
- `PA = {PFN[21:0], VPO}`
- Since the D\$ is VI, we need the set index and block offset fields to fit inside the **VPO** field to avoid synonyms.
  - This means that S can be at most 2^6.
  - The capacity per way is at most 4 KiB.
  - The only way to increase capacity is by increasing A (e.g., $A = 8$ for a 32 KiB L1 D\$).

- Could we increase the number of sets, say from 2^6 to 2^8?
  - The set index would then be `VA[13:6] = {VPN[1:0], VPO[11:6]}`.
- To prevent synonyms, the virtual-to-physical mapping must preserve these bits.
  - This is called "page coloring" (set-associative mapping for address translation).
  - Now the capacity per way can go up to 16 KiB.
  - We can get the same 32 KiB L1 D\$ with $A = 2$.

Mixed Page Sizes ("Superpages")

- Why large pages?
  - Increase the reach of each TLB entry.
  - Reduce the number of levels in the page table hierarchy.
- Complication
  - Significant complexity in TLB design.
  - Multiple page alignments.

Page Size	Offset Bits	Translated Bits	Page Table Levels (x86-64)	TLB Reach (64 entries)
4 KiB	12	36	4	256 KiB
2 MiB	21	27	3	128 MiB
1 GiB	30	18	2	64 GiB

- Approaches
  - Multiple size-specific structures (older Intel uarch). Inefficient resource utilization possible.
  - Single shared pool for multiple page sizes (e.g., STLB in Intel Sapphire Rapids). More elastic resource allocation.
  - "Coalesced TLB" (AMD), Contiguous Bit (`FEAT_TTCNP`) (Arm): Multiple contiguous 4 KiB page translations are combined into a single TLB entry, creating an opportunistic "superpage" in the TLB.
  - Latest generation Arm64: Hierarchical, multi-level TLB structure with different translation granules (4 KiB, 16 KiB, 64 KiB). The MMU decomposes large block mapping from the L2 TLB into smaller pages when required for the L1 ITLBs or DTLBs.

Three Final Issues

DMA Consistency

- DMA and MMIO normally uses PA and bypasses cache.
  - Flush cache before device write.
  - Invalidate cache before device read.

Virtualization

- Address translation needs a two-level page walk to translate a Guest VA to a Host PA.
  - Traverse Guest Page Table (GPT).
  - For each level of the GPT, traverse the nested Extended Page Table (EPT).
  - Use of dedicated Page Walk Caches to store intermediate host-level translations.

Translation Coherence

- In multicore systems, TLB consistency is maintained by the OS through "TLB shootdowns".
  - When a mapping is changed or removed on one core, the OS must ensure that stale copies are invalidated across all cores.
  - Uses Inter-Processor Interrupts (IPIs): expensive.
  - Becomes a primary scalability bottleneck.
  - Techniques like Fast Page Recycling (FPR) mitigate this by delaying shootdowns when possible. Can reduce shootdown frequency significantly in `mmap`-heavy applications.